9.7 Regex: Lookaheads and Lookbehinds

A lookahead checks what comes next without taking those characters. It is a peek, so it is called a zero-width assertion. Note 9.5 introduces the idea. This note is the set of examples, including the backtracking trap and lookbehind.

1. The notation

Syntax	Name	Meaning
`X(?=Y)`	Positive lookahead	X followed by Y
`X(?!Y)`	Negative lookahead	X not followed by Y

Read `(?=` as "is followed by" and `(?!` as "is not followed by." The `=` is a yes check. The `!` is a no check.

2. Why not just match Y?

On `"50 dollars"`:

```
str_extract("50 dollars", "\\d+ dollars")
str_extract("50 dollars", "\\d+(?= dollars)")
```

```
[1] "50 dollars"
[1] "50"
```

The first pattern keeps the word. The second requires the word and does not keep it. Later pieces of a pattern can keep matching from that same position, because the lookahead did not move forward.

3. Positive lookahead

A number only when `dollars` follows:

```
x <- "50 dollars, 20 euros, 75 dollars"
str_extract_all(x, "\\d+(?= dollars)")
```

```
[[1]]
[1] "50" "75"
```

`20` is followed by `euros`, so it is left out.

The username before `@`:

```
str_extract("ann@mail.com", "\\w+(?=@)")
```

```
[1] "ann"
```

A word only when a comma follows:

```
str_extract_all("red, green blue, pink", "\\w+(?=,)")
```

```
[[1]]
[1] "red"  "blue"
```

`green` and `pink` are not followed by a comma.

4. Negative lookahead

A `q` that is not followed by `u`:

```
str_extract_all("quick qat Iraq", "q(?!u)")
```

```
[[1]]
[1] "q" "q"
```

The `q` in `quick` is followed by `u`, so it fails. The `q` in `qat` and the `q` in `Iraq` are not.

A string that does not contain `error`:

```
x <- c("all good", "an error here", "fine")
str_detect(x, "^(?!.*error)")
```

```
[1]  TRUE FALSE  TRUE
```

`^` is the start. `(?!.*error)` asks whether `error` appears anywhere later. If it does, the check fails and the string is rejected. `all good` and `fine` pass.

A file name that is not `.txt`:

```
files <- c("a.csv", "b.txt", "c.R")
str_subset(files, "^(?!.*\\.txt$)")
```

```
[1] "a.csv" "c.R"
```

`.*\\.txt$` is “anything, then a literal `.txt` at the end.” The negative lookahead rejects that shape. `b.txt` is out. `a.csv` and `c.R` stay.

5. A trap: the engine backtracks

```
str_extract("50 dollars", "\\d+(?! dollars)")
```

```
[1] "5"
```

That is not `NA`. The engine first tries `50`. The next text is `dollars`, so the negative lookahead fails. It then gives back a digit and tries `5`. The next character is `0`, not `dollars`, so the check passes and the match is `"5"`.

Require a whole number:

```
str_extract_all("50 dollars 20 euros", "\\b\\d+\\b(?! dollars)")
```

```
[[1]]
[1] "20"
```

`\\b` is a word boundary, so `50` is one number. It is followed by `dollars`, so it fails. `20` is a whole number not followed by `dollars`.

The same idea, forbidding another digit instead of using a boundary:

```
str_extract_all("50 dollars 20 euros", "\\d+(?!\\d)(?! dollars)")
```

```
[[1]]
[1] "20"
```

`(?!\\d)` means "not followed by another digit," so the engine cannot stop in the middle of `50`.

6. Several lookaheads at once

Lookaheads at the same position must all be true. This is the usual password check:

```
pw <- c("Passw0rd123", "password", "PASSWORD1", "Sh0rt")

str_detect(pw, "^(?=.*\\d)(?=.*[A-Z])(?=.*[a-z]).{8,}$")
```

```
[1]  TRUE FALSE FALSE FALSE
```

Read from the start:

- `(?=.*\d)` means a digit appears somewhere ahead.
- `(?=.*[A-Z])` means a capital appears somewhere ahead.

- `(?=.*[a-z])` means a lowercase letter appears somewhere ahead.
- `.{8,}` then takes 8 or more characters.

Each lookahead starts over at `^`, so they are separate checks. `password` has no digit and no capital. `PASSWORD1` has no lowercase letter. `Sh0rt` is shorter than 8.

7. Lookahead or a capturing group

On `"50 dollars"`:

Goal	Pattern	Result
Keep the word in the match	`\d+ dollars`	`"50 dollars"`
Keep the whole match, and also save the number	`(\d+) dollars`	whole `"50 dollars"`, group 1 `"50"`
Match only the number	`\d+(?= dollars)`	`"50"`

```
str_match("50 dollars", "(\\d+) dollars")
str_extract("50 dollars", "\\d+(?= dollars)")
```

```
     [,1]         [,2]
[1,] "50 dollars" "50"

[1] "50"
```

A lookahead is useful in `str_replace_all()` when the checked text should stay put, because that text is not part of the match and so it is not replaced.

```
str_replace_all("50 dollars, 20 euros", "\\d+(?= dollars)", "[\\0]")
```

```
[1] "[50] dollars, 20 euros"
```

`\\0` is the whole match, which here is only the number. The word `dollars` is still there.

8. Lookbehind

A lookahead looks to the right. A lookbehind looks to the left.

Syntax	Meaning
`(?<=Y)X`	X preceded by Y
`(?<!Y)X`	X not preceded by Y

```
str_extract_all("cost $50, qty 20", "(?<=\\$)\\d+")
```

```
[[1]]
[1] "50"
```

`(?<=\\$)` checks that a `$` is immediately before the digits, and it does not take the `$`. `20` has no `$` before it.

In stringr the lookbehind must have a bounded length. `(?<=\\d+)` is an error: `Look-Behind pattern matches must have a bounded maximum length`. Use a fixed length, or a range such as `\\d{1,3}`, not `*` or `+`.

9. Key points

1. `(?=...)` means "must be followed by."
2. `(?!...)` means "must not be followed by."
3. Both are zero-width. They test, and they do not take the text.
4. Put the lookahead after the text you want to keep: `X(?=Y)`.
5. Put it at the start, `^(?=...)`, when the rule is about the whole string.
6. `\\d+(?!...)` can succeed on a prefix after the engine backtracks. Add `\\b`, or `(?!\\d)`, so the number has to finish.
7. `(?<=Y)` looks left. The text it checks must have a bounded length.